BusyBox `sort` in biocontainers: Galaxy jobs that succeed with missing data

Richard Burhans
Penn State University ORCID 0009-0006-5949-0958

Technical note, 28 October 2026

Abstract

Most single-package biocontainers supply `sort` from BusyBox, which holds its entire input in memory and silently ignores the options that make GNU `sort` spill to disk. When a job's memory limit kills such a `sort` in the middle of a shell pipeline, the next command receives an empty or truncated stream, and the pipeline still exits zero. Galaxy adds `set -e` to tool scripts but not `pipefail`, so the kill does not reach the job's exit status. We reproduced this on usegalaxy.org and two other Galaxy servers. When Slurm's memory limit killed `sort`, the job finished green with none of its 16.4 million lines in the output, even though Slurm reported the kill on the job's standard error. Galaxy's Slurm runner does not recognise that message. We measure BusyBox `sort`'s memory use, which depends on line length in a way no simple multiple of the input captures, and give five mitigations. The simplest is to sort to a file as a command of its own.

1 The problem

A Galaxy tool's command is a shell script run inside the tool's container. Biocontainers built from a single Bioconda package start from a minimal base image whose basic utilities, `sort` among them, are the BusyBox multi-call binary. In `quay.io/biocontainers/ucsc-maftobigmaf:482-h0b57e2e_0`, `/usr/bin/sort` is BusyBox 1.36.1 (812,456 bytes, 380 hard links), and so is the `sort` of all 23 `ucsc-*` version 482 images we checked (identical binary) and of the `blast`, `samtools` and `sourmash` images. Images whose packages pull in `coreutils`, such as `multiz` and `kegalign-full`, have GNU `sort` first on the path instead.

BusyBox `sort` has no external merge. It reads every line into memory, sorts, and only then writes (`coreutils/sort.c`). It accepts GNU's `-S` (buffer size) and `-T` (temporary directory) and ignores them: sorting 1,000,000 lines peaked at 83.4 MB without them and 83.8 MB with `-S 1M -T /tmp/spill`, and no file was written to the temporary directory. The long forms (`--buffer-size`, `--temporary-directory`, `--parallel`) are rejected with exit status 2. A tool author who adds `-S` and `-T` to stay within a memory limit gets no error and no protection. In BusyBox 1.32.1 the same short options were rejected instead, which produced empty output files in Bismark [1].

2 How much memory

Peak memory grows with the number of lines as well as their bytes (Figure 1). We sorted 50 MB of fixed-width numeric lines at 25 line lengths and read the peak from the container's control group.

Lines of up to 112 bytes cost their own length plus 24 to 40 bytes. Lines of 120 to 256 bytes all cost about 275 bytes each, whatever their length, so 120-byte lines need 2.5 times the input. Short lines are worst in ratio: 8-byte lines needed 6.1 times the input. BusyBox reads each line into a buffer that grows in 256-byte steps and is then trimmed (`libbb/get_line_from_file.c`), which likely explains the step, but we did not trace the allocator.

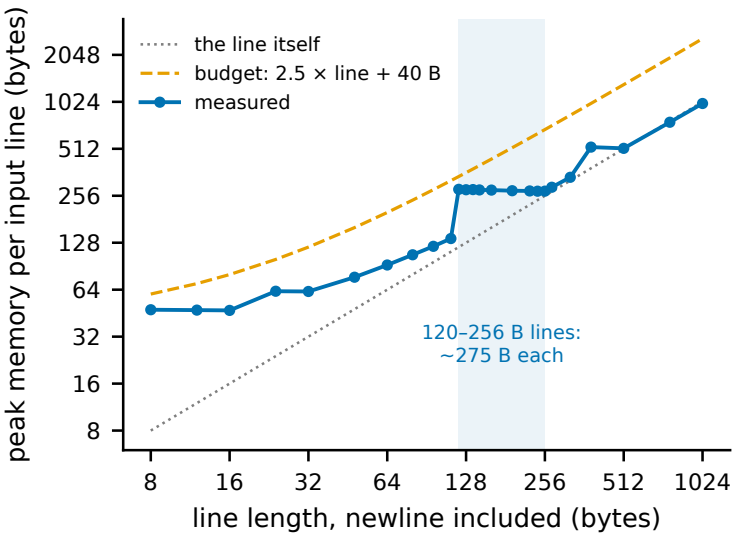


Figure 1: BusyBox `sort`'s memory per line is not a fixed multiple of the line. Peak memory per input line (control-group peak minus an empty container's 5.9 MB, divided by the line count) for 50 MB of input at each line length. Blue: measured. Grey dotted: the line itself. Orange dashed: the budget $2.5\times$ line $+40$ bytes, which covers every length measured. Shaded: lines of 120 to 256 bytes, which cost about 275 bytes each.

A budget that covers all 25 lengths is $6\,\mathrm{MB} + 2.5 \times \mathrm{input} + 40\,\mathrm{bytes} \times \mathrm{lines}$. A rule stated as a multiple of the input alone undersizes short-line data such as identifier lists or three-column BED by half or more.

3 How the kill disappears

A shell pipeline's exit status is that of its last command. When `sort` is killed mid-pipe, the consumer after it sees end of input and exits normally. Table 1 shows what reached the consumer.

If `sort` is killed while it reads, nothing has been written and the output is empty. If it is killed while it writes, because some other process in the job grows, the output is cut at an arbitrary line. Either way, without `pipefail`, the shell reports success. Which process the kernel kills is its own choice: with a live producer in the same pipe, Docker's out-of-memory handling ended the whole container in every trial, which is loud.

The Galaxy rows are the result that matters. All three servers run Galaxy 26.1. On laila, a local server running the job in Singularity, the job finished in state `ok` with exit code 0 and empty standard error, and its output held 242,248 of 2,000,000 lines. On usegalaxy.org and vgp.usegalaxy.org, which run jobs under Slurm, the job was given 512 MiB and asked to sort 16,434,823 lines. Slurm enforced the limit (peak 536.8 MB, one out-of-memory kill in the job's control group), `sort` died while reading, and the consumer received nothing. Every one of these jobs finished in state `ok` with exit code 0, and its output dataset held the single line "0". Slurm did notice: each job's standard error reads `slurmstepd: error: Detected 1 oom-kill event(s) in StepId=.... Some of your`

Table 1: A killed `sort` leaves the pipeline's exit status at zero. Each row is `... | sort | awk 'END{print NR}'`. Docker rows: `docker --memory` with swap disabled.

Setting	What killed `sort`	Exit status	Lines reaching the consumer
Docker, 64 MB, 2 M lines from a file	its own growth while reading	0	0 of 2,000,000
Docker, same, with `pipefail`	its own growth while reading	137	0 of 2,000,000
Docker, 200 MB, 60 MB allocated by another process once output began	another process's growth	0	401,222 of 2,000,000
Galaxy (laila, Singularity), SIGKILL once output began	a signal	0, job green	242,248 of 2,000,000
usegalaxy.org and vgp (Slurm, 512 MiB), input from a file	its own growth while reading	0, job green	0 of 16,434,823
usegalaxy.org and vgp (Slurm, 512 MiB), live producer	its own growth while reading	0, job green	0 of 16,434,823
usegalaxy.org and vgp (Slurm), SIGKILL once output began	a signal	0, job green	235,008 and 262,656 of 2,000,000

`processes may have been killed by the cgroup out-of-memory handler.` Galaxy did not act on it.

4 Why Galaxy does not catch it

Galaxy prefixes a tool's script with `set -e` when the tool is strict, which is the default for XML tools with `profile` 20.09 or later and always the case for YAML and user-defined tools. It never adds `pipefail`. Under `set -e` a pipeline fails only if its last command fails. A proposal to add `pipefail` to every tool script was discussed in 2018 to 2020 and closed, partly because `pipefail` "is not POSIX so it may not work when executing jobs on containers (which use `sh` instead of `bash`)" [2]. Galaxy does run containerised scripts as `/bin/sh`, but in every biocontainer we checked, `/bin/sh` is bash. `detect_errors="exit_code"` sees only the exit status, and a SIGKILL writes nothing to standard error, so `detect_errors="aggressive"` would not catch it either.

Galaxy's Slurm runner has its own out-of-memory detection, and it misses this case for two reasons (`lib/galaxy/jobs/runners/slurm.py`, lines 29–42 and 210–226). It looks for Slurm's older wording, `Exceeded job memory limit`, not the `Detected N oom-kill event(s)` that current Slurm writes. It fails a job outright when Slurm reports the state `OUT_OF_MEMORY`, and reads the job's standard error for a memory message only when Slurm reports the job as cancelled. Since Galaxy marked these jobs `ok`, Slurm evidently reported neither for a job whose script exited 0. The message also lands in the job's standard error rather than the tool's, so a tool's own standard-error checks would probably not see it (we did not test this).

5 Mitigations

1. **Sort to a file, as a command of its own.** `sort -k1,1 -k2,2n -o sorted.bed in.bed`. A kill then sets the exit status of a top-level command, which `set -e` sees. This needs no `pipefail` and has no false failures.
2. **If the sort must stay in a pipe, set `pipefail` inside the script that runs it.** A child shell does not inherit it, so a tool that runs a configuration-file script needs it in that script. Do not follow the sort with a consumer that stops early: under `pipefail`, `sort | head -1` exits 141 when `head` closes the pipe, the failure Galaxy's own "Show beginning" tool hit in 2021 [3].
3. **Use GNU `sort` for large inputs** by adding a `coreutils` requirement, so that the tool runs in a multi-package container with GNU `sort` first on the path [4]. GNU `sort` spills to disk: in one trial it finished 2,000,000 lines under a 64 MB limit. User-defined tools resolve no requirements, so for them only the container itself decides.
4. **Lint for it.** A rule that flags `sort` anywhere but last in a pipeline, unless the tool requires `coreutils`, would reach every tool author through `planemo lint`. We wrote a prototype that parses each tool with Galaxy's own parser. On the 2,061 tools in `galaxyproject/tools-iuc` (commit `ba9ee7d1`) it flags 6, 5 of them without a `coreutils` requirement. Only one sorts input whose size grows with the data: the Ensembl VEP wrapper's sort of a user-supplied annotation file. We have not tested whether that tool fails.
5. **Teach Galaxy's Slurm runner the current message.** Treating `Detected N oom-kill event(s)` in a job's standard error as a failure, whatever the exit status, would have failed every Slurm job in Table 1 except the SIGKILL control. It fixes every tool at once on Slurm servers, though not on servers that run jobs another way.

Where none of these applies, size the job with the budget above, and check that a sort's output has as many lines as its input wherever that count cannot change (not with `sort -u`).

6 Prior reports

BusyBox `sort`'s in-memory design is known: an nf-core module failed on large inputs for that reason in 2026 [5], loudly, because the calling program checked the result. A coreutils requirement was added to the tools-iuc `vcfsort` wrapper because BusyBox `sort` lacks `-V` [4]. We found no report of a mid-pipe kill producing a successful job with missing data, and none of `-S` and `-T` being silently ignored (GitHub issue search, 28 October 2026, queries listed in the repository).

Methods and availability

All measurements, scripts and raw outputs are at github.com/richard-burhans/technical-notes under `busybox-sort/`. Container measurements: `probe/probe_container.py`, Docker with cgroup v2, image `quay.io/biocontainers/ucsc-maftobigmaf:482-h0b57e2e_0`. Galaxy measurements: `probe/probe_galaxy.py`, which registers a user-defined tool (`probe/busybox_sort_probe.gxtool.yml`) and records each job's state, exit code, streams, metrics and output (`measurements/galaxy_runs.jsonl`). Galaxy runs: laila with tool version 0.1.0, usegalaxy.org and vgp.usegalaxy.org with version 0.2.0, all Galaxy 26.1.2.dev0, 28 October 2026. Galaxy source citations are to `galaxyproject/galaxy` at `3d263f95ed` (`lib/galaxy/jobs/command_factory.py`, lines 210–216). Lint prototype: `probe/check_midpipe_sort`

This note, its figure and its code are dedicated to the public domain under CC0 1.0.

References

[1] BioContainers/containers issue 483, "bismark container Linux sort version" (2022). https://github.com/BioContainers/containers/issues/483

[2] galaxyproject/galaxy issue 5884, "pipefail and errexit in all tool_scripts" (2018–2022). https://github.com/galaxyproject/galaxy/issues/5884

[3] galaxyproject/galaxy issue 12093, "Show beginning1 fails on any file longer than the number of lines requested" (2021), fixed in pull request 12112. https://github.com/galaxyproject/galaxy/issues/12093

[4] galaxyproject/tools-iuc pull request 3920, "Add coreutils requirement to vcfsort" (2021). https://github.com/galaxyproject/tools-iuc/pull/3920

[5] nf-core/modules issue 13033, "hmmer/eslsfetchindex: fails on large FASTA in containers (BusyBox sort in biocontainers image)" (2026). https://github.com/nf-core/modules/issues/13033